CS162 Assignment 1: Structs Review "The Big Day," Bird Watching

It's time to get out your tripods, high-powered cameras and 12x binoculars. Get out there and spot the rare ones! For this assignment, write a C++ program that keeps track of the birds that have been spotted. Load the birds information into memory from a text file, so that they can be manipulated in memory (printed, searched, inserted, and removed (depending on the version of the assignment)). When the program starts, your program will ask the user to enter the name of an input file, which your program will attempt to open. If the file opens, the bird information will be loaded into memory using an array of structs. If the file doesn't open, allow the user to try another file name or type "quit" or "exit" if they don't have an input file.

Quick Links: What Not to Do, Overview, Requirements, Version Requirements (A, B, C), Example Bird Lines, Sample Run, Additional Details, Strategies, Before you Get Started.

For your convenience, there is a semicolon-separated text file that can be found with this document on D2L that contains a list of birds. You may modify it or create your own list if you prefer, but it must follow the same format (see below for details).

To allow you to balance school and other priorities, the assignments for this class are separated into minimum requirements for a particular grade: A, B or C. When you read the requirements and task sections of this document, please make note of the different requirements to achieve the grade you want. You can also use those requirements to refine different versions of your assignment. By which I mean, finish the requirements for the C version before moving on to the B version and then the A version. Along with the code files for this assignment, you must complete and upload a lab from the Zybooks chapter (zylab 15.9) and complete a design document. See below for more details.

What Not to Do

One of the main topics we are going to learn this term is how to manage memory. Many class objects and functions have been written that deal with memory management automatically, so we can't use them. You must not use any of the following libraries, objects, or functions:

- **You may not use strings. Do not include the <string> library or create string variables.** Use c-strings, which are null-terminated character arrays, instead.
- **You may not use <vector> or any other stl container classes:** <array> , <list>, etc.
- **You may not use ranges or any functions associated with container libraries**: <ranges>, <algorithm>, <numeric>, <execution>, <functional>, etc.
- **You may not use any libraries that include container classes,** such as <sstream> (string streams include the string object).
- **You may not use any of the Windows specific versions of the c-string manipulation functions**, such as strcpy_s(), strncpy_s(), or any other version that ends with _s. There are many ways to use the standard <cstring> functions with Visual Studio, but probably the easiest is just to use a #pragma directive: `#pragma warning (disable: 4996)`
- **You may not use a sorting algorithm, either from a library or one that you write from scratch.** The birds must be maintained in sorted order by name. You must insert the birds in a sorted way when they are read (from the file or from the keyboard). See below and ZyBook chapter 15.7 for more details.
- **All of these rules also apply to lambdas** (you may not use lambdas with stl objects).
- **Do not use c-style input and output,** such fscanf, fprintf, etc. from the <stdio.h> library.
- **Do not use memmove() or memcpy().** Please use strcpy(), strncpy(), strcmp(), strlen(), etc.
- **Do not hardcode the name of the input file.** Ask the user for the file name.
- **Do not use global variables.** Global constants are ok.
- **Do not use integers for menu choices.** Use the characters listed below.

Assignment Overview

When the program starts, ask the user for the name of the input text file. If the file does not open and/or the user enters "quit" or "exit" for the filename, exit the program. See the sample output below for more details.

Each line of the input text file contains the information for one bird. When the program starts, load the information for each bird into a struct, and then place the struct in an array. An example of the struct format is specified in a textbox below, in the Requirements section.

The input text file uses a semicolon to separate the values.

Do not assume that there are a certain number of birds listed in the file. Stop reading data from the file when the end-of-file marker is reached or there is no more room in the array (do not write data beyond the end of the array).

Once all of the data is loaded from the file, close it and only manipulate the data in memory (only read from the file once, when the program starts).

Write all of the data in memory to an output file when the user exits the program. You may use the same name as the input file or ask the user for an output file name. Do not write data to an output file at any other time (only write to an output file at program termination).

The birds must be inserted into the array in sorted order by name. You may not use a separate sorting algorithm. In other words, you may not add the new bird to the end of the array and then sort the array. You must find the proper location for the new bird, move the other birds up one location if necessary, and then insert the new bird at the correct index (see ZyBook chapter 15.7 and below for details).

Once the data is loaded, your program will enter a menu-driven loop to allow the user to decide what to do. There are different menu options for the different versions of the assignment. See below for the separate grade requirements. **You may not use numbers for the menu options; you must use the letters listed in the example output.** When the user chooses to quit and the program terminates, you have two options: you may open the same file and write the data back to the original file so that it will be updated with any new information, or you may ask the user for a new output filename and write to that.

You must use at least one header file and multiple code implementation files for this assignment. To design the assignment, we will separate the executable code (function definitions) from the code declarations (non-executable statements such as constants, prototypes, the struct definition, etc.). Place the function definitions in an implementation file or files that use the extension .cpp. For the code declarations, place the code in a header file or files using the extension .h or .hpp. Please create at least two implementation files, one for the main function and one for all other functions. You may create more than two implementation files if you wish, and you may have one or more header files.

All character data (words and sentences) must use c-strings, which are null-terminated char arrays.

Requirements:

```
const int STR_SIZE = 128;

struct Bird {
    char name[STR_SIZE];
    char location[STR_SIZE];
    double weight;
    int sightings;
    int rarity;
    char description[STR_SIZE];
};
```

- You must use an array of structs to store the information.
- You must insert birds from the file and birds added by the user in sorted order. You may not use a sorting algorithm, either from a library or coded by you.
- You must use c-strings (null-terminated char arrays) to store words. You may not declare string variables.
- You may use the <cstring> library for the c-string manipulation functions, such as strlen, strcpy, etc. Do not use the Windows versions ending in _s and do not use use `memmove()` or `memcpy()`.
- You must use functions, have multiple code files and a header file or files.
- You may not use any of the STL container libraries such as <vector>.
- You may not use any functions that require strings, such as <sstream> (stringstream) or any functions that require STL containers, such as the <algorithm> library.
- Do not allow your program to crash because of an index out of bounds, either for the array or for c-strings.

- Use the letters for the menu choices that are listed in this document. Do not modify the letters or use integers.
- Follow the coding style listed in the C++ style guide. Make sure you include the header file comments listed in the style guide at the top of one of your files. See the detailed requirements section below.
- Complete the zylab from chapter 15, 15. CS 161B: Structs Part II zyLabs 15.9. The 15.9 zylab must be completed with 100%, and the participation activity must be completed with 80%. Sometimes there will be challenge activities, though not for chapter 15, and those must be 70% completed. Optional labs are not required (but they are good practice). Lab 15.9 is the only required lab for chapter 15.
- Open the Algorithmic Design Document, make a copy, and follow the steps to create your algorithm.
- Your assignment will not be graded until the zyLab and design document are complete.
- Load the data when the program is started and write the data back to a file at program termination. Only load from the file at the start of the program. Only write to the file at program termination.
- After the file is loaded, the program will enter a user-controlled loop. Inside of the loop, the program will ask the user what they want to do – using the letters specified below for the menu choices.

Separate Requirements for A, B, or C versions.

- The rest of the requirements are split into 3 different versions based on the type of grade you want. Each version has the minimum requirements for a particular grade – A, B or C. The B version builds on the requirements of the C version, and the A version builds on the requirements of the B version.
  - The maximum score for the C version is 2 out of 4
  - The maximum score for the B version is 3 out of 4
  - The maximum score for the A version is 4 out of 4

- The separate requirements below for the different versions require a bit of explanation. When the user wants to add a new bird to the array, they will have to specify values for the weight in ounces, the number of sightings, and the rarity. The ranges are as follows:
  - The lightest bird in the world is the Bee Hummingbird which weighs around 0.08 ounces on average, so all birds must weigh more than 0.05 ounces.
  - The ostrich can weigh over 300 pounds, and the cassowary and the emu can weigh over 100 pounds, so let's say they can't be in the competition. The next (fourth) heaviest bird is the emperor penguin, which can weigh 100 pounds, which is 1600 ounces. So let's make that the upper limit for ounces for any bird.
  - In order for a bird to be listed, the sighting count has to be at least 1. Otherwise, if the sighting count is zero, then the bird won't be listed. Let's assume that 100 people have registered for the competition in your local area, so 100 will be the maximum value for sightings.
  - Rarity scores are from 1 to 10, 1 being very common and 10 being very rare.
- One of the items in the style guide is to not use "magic numbers" when doing comparisons. Always use constants for comparing values in a range. Here are a few to give you the idea:

```
const int MAX_COUNT = 100;
const double MAX_WEIGHT = 1600.0;
```

Here are the requirements for each version (separate error checking requirements are listed after):

Version C:

- Load the birds from the file in sorted order by name.
- Print all the birds, one per line, using line numbers.
- Add a new bird to the database in sorted order by name.
  - Make sure the weight is greater than 0.05.
  - Make sure the sightings and rarity numbers are positive.
- Quit, and write the data to the output file.

Version B:

- Load the birds from the file in sorted order by name.
- Print all the birds, one per line, using line numbers.
- Add a new bird to the database in sorted order by name.
  - Make sure the weight is greater than 0.05 and less than or equal to 1600.0.
  - Make sure the sightings number is positive and less than or equal to 100.
  - Make sure the rarity number is between 1 and 10 inclusive.
- Remove a bird by index (new for B version).
- Quit, and write the data to the output file.

Version A:

- Load the birds from the file in sorted order by name.
- Print all the birds, one per line, using line numbers.
- Add a new bird to the database in sorted order by name.
  - Make sure the weight is greater than 0.05 and less than or equal to 1600.0.
  - Make sure the sightings number is positive and less than or equal to 100.
  - Make sure the rarity number is between 1 and 10 inclusive.
- Remove a bird by index.
- Print all birds for a specified location. (new for A version).
- Quit, and write the data back to the file.

NOTE: There are different error checking requirements when adding a new bird from the keyboard or removing a bird for each one of the grade versions:

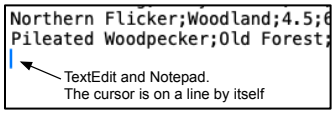
- C version
  - Check to make sure the weight is greater than 0.05.
  - Check to make sure the sightings and rarity values are positive.
- B version
  - Check to make sure the weight is greater than 0.05 and less than or equal to 1600.0.
  - Check to make sure the sightings value is positive and less than or equal to 100.
  - Check for proper array index to remove (not out of bounds – too low or too high).
- A version – All the error checks for the B version and check for input failure (don't allow the program to crash or go into an infinite loop when letters are typed for input to number variables).

See the Additional Programming Details section for more details.

The file format lists all the information for one bird on one line, and includes:

1. Name (c-string)	2. Location (c-string)
3. Weight (double)	4. Sightings (int)
5. Rarity (int)	6. Description (c-string)

Make sure that there is a newline (return) at the end of every line, including the last line. Vim and vi will automatically make sure that your text files have a newline at the end of the last line, but you must add the newline to the end if you use TextEdit or Notepad. The function that reads data from the file must be able to handle the newline at the end of the file. If it's not handled correctly, then an empty struct or a duplicate will be added to the array. See the programming strategies section for more information.



Follow the table above for the order of data items for each bird. One line in the data file has all the information for one bird. The order is as follows: name, location, weight, sightings, rarity, and description. The name, location and description must be stored as c-strings. The sightings and rarity must be stored as integers, and the weight must be stored as a double.

Please see the Additional Programming Details below for more information.

The first 10 Birds from the birds.txt file:

```
Species;Location;Weight_ounces;SightingCount;RarityScore;Description
American Robin;Backyard Feeder;2.7;14;2;Common songbird with a reddish-orange breast and cheerful call.
Northern Cardinal;Forest Edge;1.6;8;3;Bright red bird with a distinctive crest and whistle-like song.
Blue Jay;Oak Grove;3.1;11;2;Intelligent blue bird known for its loud calls and bold behavior.
American Goldfinch;Meadow;0.5;20;3;Small yellow finch often found feeding on seeds in meadows.
Mourning Dove;Farm Field;4.2;16;2;Gentle gray bird recognized by its soft cooing sounds.
Red-tailed Hawk;Grassland;38.0;3;6;Large soaring hawk with a broad wingspan and reddish tail.
Bald Eagle;River;160.0;2;8;Majestic bird of prey with a white head and powerful beak.
Great Blue Heron;Wetland;80.0;4;5;Tall wading bird that hunts fish in wetlands and streams.
Mallard Duck;Pond;38.5;18;1;Familiar duck with a green-headed male and mottled brown female.
Canada Goose;Lake Shore;120.0;13;1;Large waterfowl famous for flying in V-shaped formations.
```

Notice that the first line of the text file has the column header names. You can strip that first line out if you want and save the file without it, or you can read the first line in your program and discard it. There is an example file with 20 birds here on Google and attached to D2L.

Sample Runs / Example Output

The examples below have all the options, which is the A version of the assignment. Input is bold (for this document only). For the C version, you will only present the options to Add, Print, and Quit. Any other options should generate an error message and be ignored. The menu options for the C version are:

```
What would you like to do? (P)rint the list, (A)dd, or (Q)uit?
```

For the B version, you will have the options to Add, Print, Remove and Quit:

```
What would you like to do? (P)rint the list, (A)dd, (R)emove a bird, or (Q)uit?
```

Example 1, exit for the file name.

```
Time for some bird spotting!
What is the file name to open? ('exit' if you don't have one): notThere.txt
File notThere.txt did not open. Please try again or type 'exit': exit
Exit was entered, so no birds were loaded.
Keep those cameras ready! Until next time.
```

Example 2, valid input file.

```
Time for some bird spotting!
What is the file name to open? ('exit' if you don't have one): notThere.txt
File notThere.txt did not open. Please try again or type 'exit': birds.txt
10 birds were successfully loaded.
The birds in the database are:

    Name              Location    oz. Sights Rare  Description
    ----              --------    --- ------ ----  -----------
 1: American Goldfinch  Meadow    0.5    20     3  Small yellow finch often found feeding on seeds in meadows.
 2: American Robin      Varied    2.7    14     2  Common songbird with a reddish-orange breast and cheerful call.
 3: Bald Eagle         Wetland  160.0     2     8  Majestic bird of prey with a white head and powerful beak.
 4: Blue Jay            Forest    3.1    11     2  Intelligent blue bird known for its loud calls and bold behavior.
 5: Canada Goose       Wetland  120.0    13     1  Large waterfowl famous for flying in V-shaped formations.
 6: Great Blue Heron   Wetland   80.0     4     5  Tall wading bird that hunts fish in wetlands and streams.
 7: Mallard Duck       Wetland   38.5    18     1  Familiar duck with a green-headed male and mottled brown female.
 8: Mourning Dove       Meadow    4.2    16     2  Gentle gray bird recognized by its soft cooing sounds.
 9: Northern Cardinal   Forest    1.6     8     3  Bright red bird with a distinctive crest and whistle-like song.
10: Red-tailed Hawk     Meadow   38.0     3     6  Large soaring hawk with a broad wingspan and reddish tail.

What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? a
What is the new bird's name? Pigeon
What is the location of the new bird? Urban
How much does the new bird weigh in ounces? About 10 or so
Please enter a weight between 0.05 and 1600.0: -10.5
Please enter a weight between 0.05 and 1600.0: 10.5
How many sightings are there: Millions
Please enter a number between 1 and 100: 90
How rare on a scale of 1 to 10 is the new bird? super common
Please enter a number between 1 and 10 inclusive: 1
Please provide a short description of the new bird: Adaptable gray/iridescent bird commonly found in cities.
The new bird was successfully added to the list.

What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? B
B is not a valid option. Please try again.

What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? P
The birds in the database are:

    Name              Location    oz. Sights Rare  Description
    ----              --------    --- ------ ----  -----------
 1: American Goldfinch  Meadow    0.5    20     3  Small yellow finch often found feeding on seeds in meadows.
 2: American Robin      Varied    2.7    14     2  Common songbird with a reddish-orange breast and cheerful call.
```

```
 3: Bald Eagle         Wetland  160.0     2     8  Majestic bird of prey with a white head and powerful beak.
 4: Blue Jay            Forest    3.1    11     2  Intelligent blue bird known for its loud calls and bold behavior.
 5: Canada Goose       Wetland  120.0    13     1  Large waterfowl famous for flying in V-shaped formations.
 6: Great Blue Heron   Wetland   80.0     4     5  Tall wading bird that hunts fish in wetlands and streams.
 7: Mallard Duck       Wetland   38.5    18     1  Familiar duck with a green-headed male and mottled brown female.
 8: Mourning Dove       Meadow    4.2    16     2  Gentle gray bird recognized by its soft cooing sounds.
 9: Northern Cardinal   Forest    1.6     8     3  Bright red bird with a distinctive crest and whistle-like song.
10: Pigeon              Urban    10.5    90     1  Adaptable gray/iridescent bird commonly found in cities.
11: Red-tailed Hawk     Meadow   38.0     3     6  Large soaring hawk with a broad wingspan and reddish tail.

What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? L
For what location would you like to print the birds? Tundra
There are no birds from the Tundra in the database.
What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? l
For what location would you like to print the birds? Forest
The birds from the Forest are:
1: Blue Jay            Forest    3.1    11     2  Intelligent blue bird known for its loud calls and bold behavior.
2: Northern Cardinal   Forest    1.6     8     3  Bright red bird with a distinctive crest and whistle-like song.

What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? S
What bird by name would you like to find? Archeopteryx
The bird Archeopteryx was not found in the database.
What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? s
What bird by name would you like to find? Bald Eagle
The information for the Bald Eagle is:
Location: Wetland, Weight: 160.0 oz, Sightings: 2, Rarity: 8, Description: Majestic bird of prey with a white head and
powerful beak.
What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? R
What bird by index, starting at 1 and up to and including 11 would you like to remove? 0
That index is invalid or out of bounds, please try again: Eleven
That index is invalid or out of bounds, please try again: 20
That index is invalid or out of bounds, please try again: 8
The bird at index 8 was removed.

What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit? P
The birds in the database are:

    Name              Location    oz. Sights Rare  Description
    ----              --------    --- ------ ----  -----------
 1: American Goldfinch  Meadow     0.5     20     3  Small yellow finch often found feeding on seeds in meadows.
 2: American Robin       Varied     2.7     14     2  Common songbird with a reddish-orange breast and cheerful call.
 3: Bald Eagle         Wetland  160.0      2     8  Majestic bird of prey with a white head and powerful beak.
 4: Blue Jay            Forest    3.1     11     2  Intelligent blue bird known for its loud calls and bold behavior.
 5: Canada Goose       Wetland  120.0     13     1  Large waterfowl famous for flying in V-shaped formations.
 6: Great Blue Heron   Wetland   80.0      4     5  Tall wading bird that hunts fish in wetlands and streams.
 7: Mallard Duck       Wetland   38.5     18     1  Familiar duck with a green-headed male and mottled brown female.
 8: Northern Cardinal   Forest    1.6      8     3  Bright red bird with a distinctive crest and whistle-like song.
 9: Pigeon              Urban    10.5     90     1  Adaptable gray/iridescent bird commonly found in cities.
10: Red-tailed Hawk     Meadow   38.0      3     6  Large soaring hawk with a broad wingspan and reddish tail.

What would you like to do? (P)rint the list, (A)dd, (R)emove, (S)earch, Print by (L)ocation, or (Q)uit Q
What file name would you like to use for the output file? out.txt
The file out.txt has been written.
Keep those cameras ready! Until next time.
```

Academic Integrity

You may NOT, under any circumstances, begin a programming assignment by looking for completed code on StackOverflow or Chegg or any other website, which you can't claim as your own. Please check out the Student Code of Conduct at PCC.

You must write your code yourself. You may not collaborate with other students when writing your code. If you start with code from another source and just change the variable names or other content to make it look original, you will receive a zero on the assignment. I may ask you to explain your assignment verbally. If you cannot satisfactorily explain what your code does and why you wrote it in a particular way, then you should not expect to receive credit. Do not share your code with any other students and do not post it on a public website.

Additional Programming Details

- You may use a reasonable length for your c-strings, such as 128 or 256, declared as a global constant (remember not to use literals in your code for things like array size):
  ```
  const int STR_SIZE = 256;
  const int ARR_SIZE = 40;
  ```
- Use C++ style input and output for reading and writing from the keyboard and from a file. Except for memcpy() and memmove(), you may use the functions in <cstring>, <cctype>, <cstdlib>, <iomanip>, and <iostream>.

- Don't use extraction (>>) when reading data into c-strings from the keyboard. Use cin.getline(), both for security and to allow for whitespace (multiple words). You can specify the delimiter in order to stop reading at the semicolon: `cin.getline(str, STR_SIZE, ';');` If you leave off the delimiter it will stop at and remove the newline from the stream: `cin.getline(str, STR_SIZE);`. So the default value is the newline ('\n'). Follow this link to see more details: cplusplus.com/getline. See below in Programming Strategies for using the delimiter version for reading data items from a file.
- Create your array of structs in the main function. You must pass the array of structs from the main function to other functions as a parameter. Do not use any global variables (constant globals are OK).
- You must use functions for this assignment. There are at least four that you must create for the C-version, including: `loadBirds(), writeBirds(), addBird(), and printBirds()`. For the B version, you must add `remove()`, and for the A version, `printByLocation()` and `search()`. You may add other functions, such as `welcome(), openFile(),` etc. if you wish, and you may change the name of these functions, such as `removeByIndex()` instead of just `remove()`. These functions must be in a file other than main.cpp, and each function must have a prototype in a header file (do not place the prototypes in a code file).
- I would encourage you to use a function called `insert()`. This function can then be called by both `loadBirds()` and `addBird()`. The only difference between these two functions is where the data comes from, the file or the keyboard. In both functions I would recommend loading up a new Bird struct and then passing the count, the new struct and the array of structs to the `insert()` function. That way you only have to write the code to insert a new bird once. The prototype may look like:
```
void insert(int count, Bird toAdd, Bird birds[]);
```
- Please use only the letters listed for the menu options and don't use integers. Your program should be able to handle both upper and lowercase. You can use `toupper()` or `tolower()` from the <cctype> library to convert from one to the other: `letter = toupper(letter);`. Use 'A' or 'a' for adding an bird, 'P' or 'p' for printing the list, 'S' or 's' for searching, 'R' or 'r' for removing, 'L' or 'l' to print by location, and 'Q' or 'q' to quit. No other letters and no numbers are acceptable for menu selection.
- Make sure you check for index out of bounds before adding new data to the array. When loading data, compare the index to the ARR_SIZE constant (or whatever name you use for the size of an array). If they are equal, there is no more room, so stop loading data. Suppose you use a variable called `count` to keep track of the number of birds in the array. Count will start out at zero (the first location in the array), and then add 1 every time you add a new bird: `count++;` You can check if the array is out of room with an if statement or as part of a loop condition:
```
if(count >= ARR_SIZE) // no more room so don't load the new data.
```
There are two functions where you need to do this check: `loadBirds()` and `addBird()`. Alternatively if you have `loadBirds()` and `addBird()` call a third function such as `insert()` you can do this check there. The `insert()` function is a good idea so you won't have to duplicate your code, and will make future assignments easier.
- Check to make sure that the input file is open before reading from it, using `ifstream.is_open()`. Ask the user to try a different filename if the file does not open or have them type "quit" or "exit" to exit the program at that point (if they don't have an input file).
- Avoid using literals ("magic numbers"). Use constants instead. For example don't use: `char name[100];` Declare a constant such as STR_SIZE and use: `char name[STR_SIZE];` instead.
- Structure your functions using selection (if, else if, and else) so that there is only one return statement at the bottom of your functions. Do not terminate your functions early by placing return statements at other locations and do not use the `exit()` function. Consult the style guide for more information.
- Use proper indentation, function comments and file header comments, also found in the style guide.

- Make sure you check for negative numbers and bad data when reading number data from the user.
- Separate your project into multiple files. The preprocessor directives that should be placed around the contents of a header file are: `#ifndef`, `#define` and `#endif`. Then use `#include "functions.h"` to include the header file in your code files. You should never include code (.cpp) files in other files. Only header files and libraries should be included in other code or header files. Also notice that to include a header file, you use the double quotes around the filename (instead of <> for libraries). The two directives, `#ifndef` and `#define`, require an identifier and should be at the top of the file: `#ifndef _BIRDS_` and `#define _BIRDS_` (the identifiers must match and only letters and underscore are acceptable). `#endif` should be placed at the bottom of the header file. You can use any identifier you wish, but the identifier for both `#ifndef` and `#define` must match. All caps and underscores are by convention.
- You must place the struct definition in a header file and not in a code file, and it must have the format (number and type of the member variables) listed in the textbox above. You may change the name of the struct and member variables, such as, instead of `struct Bird`, you could use `struct BirdInfo`, for example, and you could shorten `description` to just `desc` if you prefer.
- Use descriptive variable names. `desc` is short enough to be easily typed but long enough to be descriptive. Do not use single letter variable names, except for loop counters (such as i). For example, do not use `n` for name, `d` for description, etc.
- You must have a `main.cpp` file and a separate code file for other functions such as `birdFunctions.cpp`, and a `birdFunctions.h` or `.hpp` file, but you may have other files as well. To help me streamline my grading, please use these standard names for any "helper" functions (such as `welcome()`) files:
  - `helper.h` and `helper.cpp`
  - `tools.h` and `tools.cpp`, or
  - `functions.h` and `functions.cpp`.

  Please don't use any other names for your code and header files, and don't change the capitalization.

Programming Strategies

- You may assume that the input file is formatted correctly, so no error checking is necessary when reading from the file. Once you save an output file, make sure it can be read by your program as the input file. That way you'll know the output file is formatted correctly. To test this, run your program once and save the output file. Then run the program again, but make sure you specify the output file from the first run as the input file for the second run.
- You don't have to use all of the birds in the full text file, containing 20 birds. Sometimes it's easier to trouble shoot your program using just a few lines, say 3 or 4, in the input text file.
- When reading numbers from the keyboard there are a few different ways to check for non-number input. The first method is to check for input failure, and the other is to use `atoi()` and `atof()`, found in the `<cstdlib>` library. Suppose you have a variable called `rarity` set up to take input from cin. After storing the input, use a while statement to check for input failure and values out of range:

```
const double MAX_RARE = 10; // Somewhere in the header file.
...
cout << "How rare on a scale of 1 to " << MAX_RARE << " is the new bird? ";
cin >> rarity;
cin.ignore();
while(cin.fail() || (rarity <= 0 || rarity > MAX_RARE)) {
    cin.clear();                    // clear the fail state.
    cin.ignore(LARGE_NUMBER, '\n');  // remove chars from the input buffer.
    cout << "Please enter a number between 1 and " << MAX_RARE << " inclusive: ";
    cin >> rarity;
   cin.ignore();
}
```

The other way to check for non-number input is to use `atof()` and `atoi()`, which means "ascii to float" and "ascii to int." If the input string can't be converted to a number, then atoi will return zero, so this is not a good method to check for values if zero is valid input (both zero and bad data will have a result of zero). Check for input failure and negative values by checking the result for <= 0:

```
char rareStr[STR_SIZE];
int rarity = 0;
cout << "How rare on a scale of 1 to " << MAX_RARE << " is the new bird? ";
cin.getline(rareStr, STR_SIZE);
rarity = atoi(rareStr);
while(rarity <= 0 || rarity > MAX_RARE) {
    cout << "Please enter a number between 1 and " << MAX_RARE << " inclusive: ";
    ...
}
```

By the way, don't copy-and-paste any of this code from this document into your code file. The text in this document may have non-ASCII characters (Unicode), which will cause errors during compilation.

- You can use the example database file, or you can create your own input file using your favorite text editor on the Linux system: vim, emacs, nano, etc. If you use a Windows or Mac program like Notepad++ or TextEdit, make sure that the last character in the file is a newline (return character) like the picture above (notice the cursor on the next line down), and make sure you save the input file in text file format with Unix line termination, rather than rich text, Word, or some other format. Use the `<fstream>` library to load the file. You must use C++-style input and output objects and manipulators; do not use any C-style functions for input and/or output, such as `fprintf`, `fscanf`, etc.
- When doing list output, you may use setw() and left/right from the <iomanip> library to align the data into columns, but it is not required. Use fixed, showpoint, and setprecision(1) for floating point number output.
- Do not assume that the input file contains a certain number of birds. Read from the file using an EOF-controlled loop (loop to the end-of-file marker. See below for more details).
- Do not use regular cin or ifstream extraction (>>) to read data into any c-strings, because this is a potential security issue called buffer overflow. Extraction also stops at whitespace, so you won't be able to take multiple-word input. Use `cin.getline()` or `inFile.getline()` instead. These functions can take 2 or 3 arguments: the c-string, the number of characters to read, and the delimiter (optional). If you leave off the delimiter, the default is the newline ('\n'). The other version of `ifstream.getline()` allows you to set the delimiter. For example, the semicolon: `birdsFile.getline(name, STR_SIZE, ';');` For more information, check out the cplusplus.com page.
- The input filename can be stored in main so it can be opened as an input file first, and then later as an output file if you prefer. Otherwise you can create the filename in an `openFile()` function or the `loadBirds()` function. You can either create your ifstream object in main, or wait to create it in the `loadBirds()` function. Close the file when done reading it. If you create the ifstream object in main, then you must pass it as a reference parameter to the `loadBirds()` and/or `openFile()` function. If the filename is in `main()` and you create the file in the `loadBirds()` function, then you will have to pass the filename as a c-string. Use the same name or ask the user for the output filename for an ofstream for `writeBirds()`.
- The struct array must be passed to all of the functions (`loadBirds()`, `searchBirds()`, `addBird()`, `remove()`, `printByLocation()` and `printAllBirds()`), because the array must be declared in main. So, the prototype for the `loadBirds()` function would be either:
  `int loadBirds(Bird birdArray[], ifstream & birdFile);` or:
  `int loadBirds(Bird birdArray[], const char filename[]);`
  The first version passes the file to the function by reference, and the second one passes the name as a c-string. The difference is where you decide to open the file. Notice that `loadBirds()` has an int return type. This is so it can pass back the number of birds that were loaded into the array (10 birds, in the example above). Announce the number of birds that were loaded when the function returns to main, and

store the count in a local variable, so it can be passed to the other functions. You can also use the return value to signal whether the function was successful or unsuccessful. If the `loadBirds()` function is unsuccessful for some reason, then the function could return an error code. Remember not to use literals, so set up a constant:

```
const int ERROR = -1;
```

- When testing your code for `loadBirds()`, it may appear that your program is loading the last line of the file twice or there is an empty struct. What's really happening is that your program is trying to read a line from the end of the file, after all the data has been loaded, but before the end-of-file marker has been reached. A common fix for this is to always read a new line as the last statement in a loop, and check for eof in the loop condition. This strategy may also require you to read data once before entering the loop. Another common way to check for end of file is to use a getline statement as the loop condition:

```
while (inFile.getline(myCString, STRSIZE, ';')) // false if eof().
```

The other way to check for eof() is to use a `peek()` statement near the end of the loop: `inFile.peek()`. If `peek()` encounters the end-of-file marker, then `inFile.eof()` will return true.

Before you get started:

- **Check out the video store example in zyBooks, CS161B:Section 15.6**
- **Check out the insert sorted into an array example in zyBooks, CS 161B:Section 15.7**
- Complete zyBooks section **15. CS 161B: Structs Part II zyLabs 15.9.**
- Open the Algorithmic Design Document, make a copy, and follow the steps to create your algorithm.
- **Be sure to include your screenshot of the zyBooks completion in the algorithm document. The screenshot must be present for your assignment to be graded.**
- Create your source code files, build and test your program.
- Code may be written using any development environment but must use Standard C++.
- PSU-bound students are strongly encouraged to do all development on the PCC Linux server.
- Please open and compare your work with the grading rubric before submitting.
- Remember to follow all C++ style guidelines.
- Download the algorithm document as a PDF file, include or compress it with the other code files and your txt file, and submit to the D2L assignment section.
- You may express your algorithm as pseudocode or a flowchart. Please note that your pseudocode must follow the syntax requirements shown in the design document - you may not use C++ syntax as a substitute for pseudocode.

After completing this assignment, you will be able to:

- Write code using struct-defined data types
- Organize source code in multiple header and implementation files (.h and .cpp files)
- Create, initialize, read, update, and display the contents of struct variables
- Use functions with struct parameters and return types
- Read from and write to an input data file
- Use C-strings to represent character strings

Additional Support

- Post a question for the instructor in the Ask Questions! area of the Course Lobby.
- Set up a Zoom session with the instructor.
- Go to the Discord server and post a question there.